

REAL WORLD SQL SCENARIOS

1. Find Duplicate Records

Scenario:

Your users table has duplicate emails. You need to find them.

Table: **users**

id | email

Query:

```
SELECT email, COUNT(*) AS count
FROM users
GROUP BY email
HAVING COUNT(*) > 1;
```

✅ Why this matters:

Duplicate records break authentication, reporting, and CRM systems.

2. Get Second Highest Salary

Scenario:

HR asks for the second highest salary from employees.

Table: **employees**

id | name | salary

Query:

```
SELECT MAX(salary) AS second_highest_salary
FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);
```

3. Find Customers Who Never Ordered



Scenario:

Marketing wants users who signed up but never placed an order.

Tables:

customers(id, name)

orders(id, customer_id)

Query:

```
SELECT c.id, c.name
FROM customers c
LEFT JOIN orders o
ON c.id = o.customer_id
WHERE o.id IS NULL;
```

💡 **Common interview favorite**

4. Total Sales Per Month

Scenario:

Finance wants monthly revenue.

Table: orders

id | order_date | amount

Query:

```
SELECT
    DATE_FORMAT(order_date, '%Y-%m') AS month,
    SUM(amount) AS total_sales
FROM orders
GROUP BY month
ORDER BY month;
```

5. Top 3 Highest Paid Employees Per Department

Scenario:

Management wants top performers per department.



Table: employees

id | name | salary | department

Query (Using Window Function):

```
SELECT *
FROM (
    SELECT *,
           DENSE_RANK() OVER (PARTITION BY department ORDER BY
salary DESC) AS rank
    FROM employees
) ranked
WHERE rank <= 3;
```

🔥 Very common in real analytics roles

6. Running Total (Cumulative Sales)

Scenario:

Dashboard needs cumulative revenue over time.

Table: orders

Query:

```
SELECT
    order_date,
    amount,
    SUM(amount) OVER (ORDER BY order_date) AS running_total
FROM orders;
```

7. Find Consecutive Login Days

Scenario:

Product team wants users who logged in 3 days in a row.

Table: logins

user_id | login_date



Query:

```
SELECT user_id
FROM (
    SELECT
        user_id,
        login_date,
        login_date - INTERVAL ROW_NUMBER() OVER (PARTITION BY
user_id ORDER BY login_date) DAY AS grp
    FROM logins
) t
GROUP BY user_id, grp
HAVING COUNT(*) >= 3;
```

💥 **Advanced real-world logic**

8. Delete Duplicate Records (Keep One)

Scenario:

Clean duplicates but keep the lowest `id`.

Table: `users`

Query:

```
DELETE FROM users
WHERE id NOT IN (
    SELECT MIN(id)
    FROM users
    GROUP BY email
);
```

⚠️ **Always test with SELECT first**

9. Find Products Never Sold



Scenario:

Inventory wants dead stock.

Tables:

```
products(id, name)
sales(product_id)
```

Query:

```
SELECT p.id, p.name
FROM products p
LEFT JOIN sales s
ON p.id = s.product_id
WHERE s.product_id IS NULL;
```

10. Get Percentage of Each Category

Scenario:

What % of sales comes from each category?

Table: `sales`

```
category | amount
```

Query:

```
SELECT
    category,
    SUM(amount) * 100.0 / SUM(SUM(amount)) OVER () AS percentage
FROM sales
GROUP BY category;
```

JOINS (INNER, LEFT, RIGHT)

Scenario 1: Find customers who placed orders

Business: Sales wants only *active customers*.

Tables:

```
customers(id, name)
```



```
orders(id, customer_id, amount)
```

Query (INNER JOIN):

```
SELECT c.id, c.name, o.amount  
FROM customers c  
INNER JOIN orders o  
ON c.id = o.customer_id;
```

- ✓ Returns only customers with orders
 - ✓ Used in revenue reports
-

Scenario 2: Find customers who never ordered

Business: Marketing wants inactive users.

Query (LEFT JOIN):

```
SELECT c.id, c.name  
FROM customers c  
LEFT JOIN orders o  
ON c.id = o.customer_id  
WHERE o.id IS NULL;
```

 Extremely common interview question

Scenario 3: Find orders without customer records

Business: Data integrity check.

Query (RIGHT JOIN):

```
SELECT o.id, o.amount  
FROM customers c  
RIGHT JOIN orders o  
ON c.id = o.customer_id  
WHERE c.id IS NULL;
```



WINDOW FUNCTIONS



Scenario 4: Rank employees by salary per department

Business: HR performance review.

```
SELECT *,  
       RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS  
rank  
FROM employees;
```

- ✓ Used in dashboards
 - ✓ Avoids complex subqueries
-



Scenario 5: Running total of daily sales

Business: Finance dashboard.

```
SELECT  
    order_date,  
    amount,  
    SUM(amount) OVER (ORDER BY order_date) AS running_total  
FROM orders;
```



Window functions = analytics superpower



AGGREGATIONS



Scenario 6: Monthly revenue report

Business: CFO wants monthly numbers.

```
SELECT  
    DATE_FORMAT(order_date, '%Y-%m') AS month,  
    SUM(amount) AS revenue  
FROM orders  
GROUP BY month;
```



Scenario 7: Average order value per customer

Business: Customer segmentation.

```
SELECT
    customer_id,
    AVG(amount) AS avg_order_value
FROM orders
GROUP BY customer_id;
```

SUBQUERIES

Scenario 8: Employees earning above company average

Business: Bonus eligibility.

```
SELECT *
FROM employees
WHERE salary > (
    SELECT AVG(salary)
    FROM employees
);
```

✓ Filters based on derived metrics

Scenario 9: Products priced higher than category average

Business: Pricing review.

```
SELECT *
FROM products p
WHERE price > (
    SELECT AVG(price)
    FROM products
    WHERE category = p.category
);
```

 Correlated subquery (advanced skill)



DATA CLEANING LOGIC



Scenario 10: Remove duplicate users

Business: Fix CRM data.

```
DELETE FROM users
WHERE id NOT IN (
    SELECT MIN(id)
    FROM users
    GROUP BY email
);
```



Always preview with `SELECT` first



Scenario 11: Normalize NULL values

Business: Reporting breaks due to NULLs.

```
SELECT
    id,
    COALESCE(phone, 'Not Provided') AS phone
FROM customers;
```



PERFORMANCE-AWARE QUERIES



Scenario 12: Avoid `SELECT *`

Business: Reduce query cost.



Bad:

```
SELECT *
FROM orders;
```



Good:



```
SELECT order_id, order_date, amount
FROM orders;
```

- ✓ Faster
 - ✓ Less memory
-

Scenario 13: Use indexes-friendly filters

Business: Query slowing down on large tables.

 Bad:

```
SELECT *
FROM orders
WHERE YEAR(order_date) = 2024;
```

 Good:

```
SELECT *
FROM orders
WHERE order_date BETWEEN '2024-01-01' AND '2024-12-31';
```

 This **actually improves performance**

Scenario 14: EXISTS vs IN

Business: Large datasets.

```
SELECT *
FROM customers c
WHERE EXISTS (
    SELECT 1
    FROM orders o
    WHERE o.customer_id = c.id
);
```

- ✓ Faster than **IN** for large tables
-